Módulo 2 - Grupo: JOULIE FARADAY

2026-04-15

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando *data()* en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo *$* despues del comando de informacion especifico que queremos.

También está el comando *summary()*, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

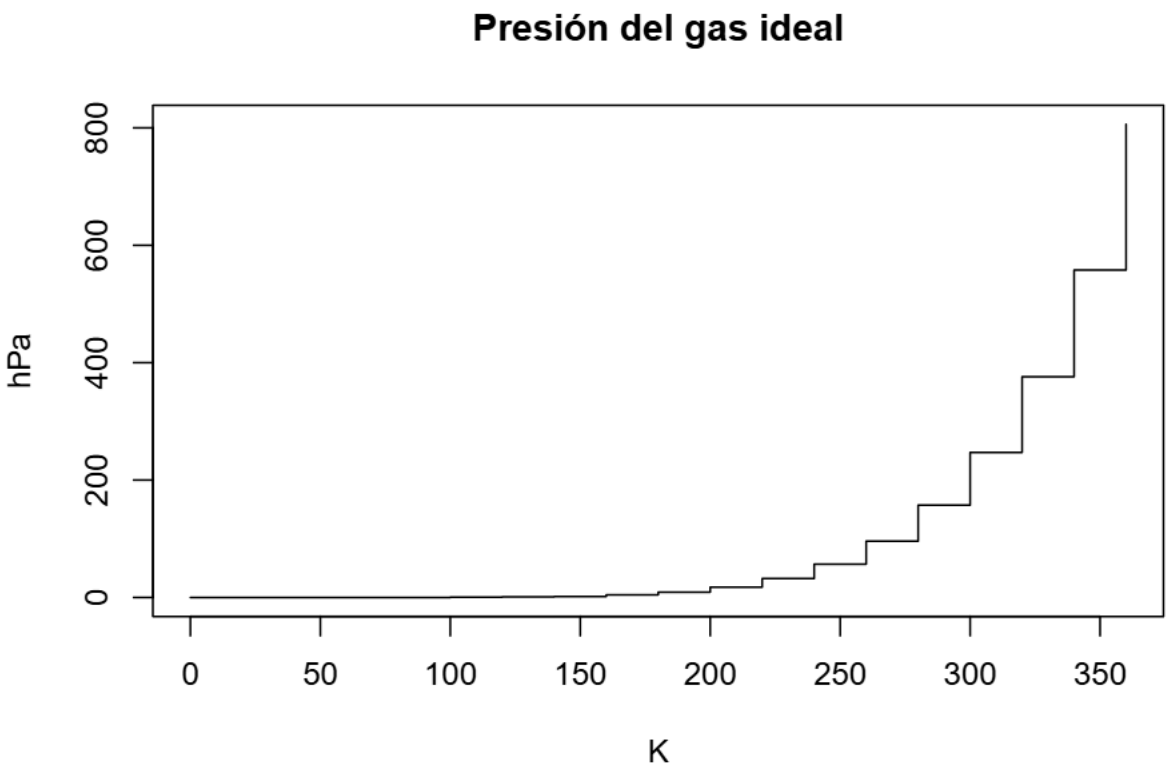
```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el "error experimental", en este caso, 5).

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 36.918633 11.448401 20.603750 28.210645  8.278640 22.479666 25.191805
##   [8] 17.866674 27.596383 15.712608 28.237284 15.515778 19.488911 12.148053
##  [15] 27.445061 20.958287 20.574622 20.299919 23.093346 33.713268 26.422907
##  [22] 26.619874 13.975526 30.267047 17.300481 30.424287 18.438108 30.762470
##  [29] 24.175796 25.374354 15.758394 16.698110 23.613447 12.701057 24.401472
##  [36] 20.009027 23.658656 23.057250 21.094253 19.192835 18.472495 18.563589
##  [43] 23.348094 10.700683 25.510989 23.113324 22.816767 17.168685 30.515121
##  [50] 29.006377 16.910635 25.297719 22.390614 31.625147 24.795019 29.097893
```

```
##  [57] 23.220179 21.106481 23.276860 17.501746 24.518033 28.616439 20.941969
##  [64] 19.867717 26.066341 25.097457 34.971706 16.424803 20.304618 31.600723
##  [71] 19.117436 14.213723 21.911756 19.736117 19.751404 20.952442 22.665587
##  [78] 22.951766 26.618838 25.315877 22.760307 12.971355 19.983351 32.191633
##  [85] 28.143142 18.076503 27.791487 17.167008 24.758516 15.854120 23.463061
##  [92] 13.082358 15.704193 26.414290 11.942104 24.120130 21.198599 22.136726
##  [99] 33.473672 25.782986 19.798957 20.453141 22.484899 18.866331 19.488612
## [106] 16.008541 26.708413 18.479646 25.283926 18.331041 22.306146 30.658570
## [113] 14.852826 17.229385 19.403740 22.880189  8.927576 16.937218 17.085555
## [120] 13.705749 31.396172 26.137511 24.614374 23.983034 14.265244 36.162585
## [127] 28.880164 22.227456 25.525008 24.734172 29.663448 30.074755 21.345098
## [134] 16.157397 15.655140 21.250705 24.278220 15.760949 16.758052 24.101254
## [141] 15.266764 25.773341 16.354299 22.213190 36.875261 20.113643 22.204517
## [148] 26.521640 22.294832 26.394624 16.807594 21.690335 25.561305 30.122547
## [155] 26.388299  5.729664 25.285897 30.442202 27.633372 25.165090 26.586527
## [162] 22.139335 16.000846 26.532333 18.228348 14.477687 22.054771 27.826611
## [169]  6.870278 21.548585 23.567244 25.147205 13.588402 18.692972 26.680209
## [176] 24.446706 31.573701 24.265581 16.325198 16.857814 14.933318 15.221775
## [183] 23.821718 21.017424 17.831882 21.094530 26.781884 26.605519 23.730375
## [190] 20.595268 20.769996 16.443490 18.134201 13.448988 30.228893 23.601866
## [197] 19.587978 25.391128 25.521151 16.503626 28.250880 19.662051 29.740916
## [204] 17.182518 13.989329 20.369584 16.467067 21.433994 38.204019 20.030099
## [211] 22.465611 17.423179 19.514993 20.963483 12.929107 33.987125 15.924185
## [218] 13.061363 26.624794 24.461127 31.599655 18.768042 31.859446 21.137632
## [225] 31.465422 22.905674 21.710807 23.661942 24.480316 26.225119 25.365740
## [232] 25.020966 17.499377 25.697855 18.053037 25.825187 26.329758 21.686024
## [239] 25.218492 32.365650 25.758445 13.759730 17.226399 17.662459 16.363546
## [246] 25.420543 20.822968 30.558113 15.070576 17.336222 24.859317 22.720028
## [253] 24.533117 24.610940  9.343306 15.404708 22.677322 22.116562 18.483681
## [260] 29.275115 16.632905 22.117087 16.861965 12.237714 20.325394 22.233513
## [267] 20.622624 18.134731 31.170093 25.473683 19.073978 24.723484 23.706351
## [274] 11.000981 13.768483 20.773948 20.435050 18.494204 26.878631 22.795093
## [281] 14.204233 18.960026 25.519103 19.531475 12.309989 25.687517 29.744990
## [288] 25.138610 27.233096 32.704649 33.190273 17.725653 25.464605 20.681886
## [295] 28.602792 23.081052 21.468455 20.058831 23.065734 19.685575 26.296494
## [302] 17.282057 28.442532 24.179435 19.332834 19.781950 23.510532 15.958120
## [309] 21.638223 21.012699 28.981978 27.779909 26.075788 22.053420 26.580395
## [316] 20.053487 18.809414 17.107087 22.383823 24.147145 15.854587 19.547017
## [323] 20.209038 18.823339 30.309682 17.219379 11.013317 29.987866 15.694130
## [330] 17.475932 21.002097 26.981853 24.425695 23.695802 26.712924 31.971252
## [337] 23.992701 10.199658 18.684867 25.271003 15.867631 21.286600 23.411908
## [344] 25.815566 23.732193 20.744162 12.791646 21.866621 17.730684 25.856786
```

```r
w1 <- length(z1)
w1
```

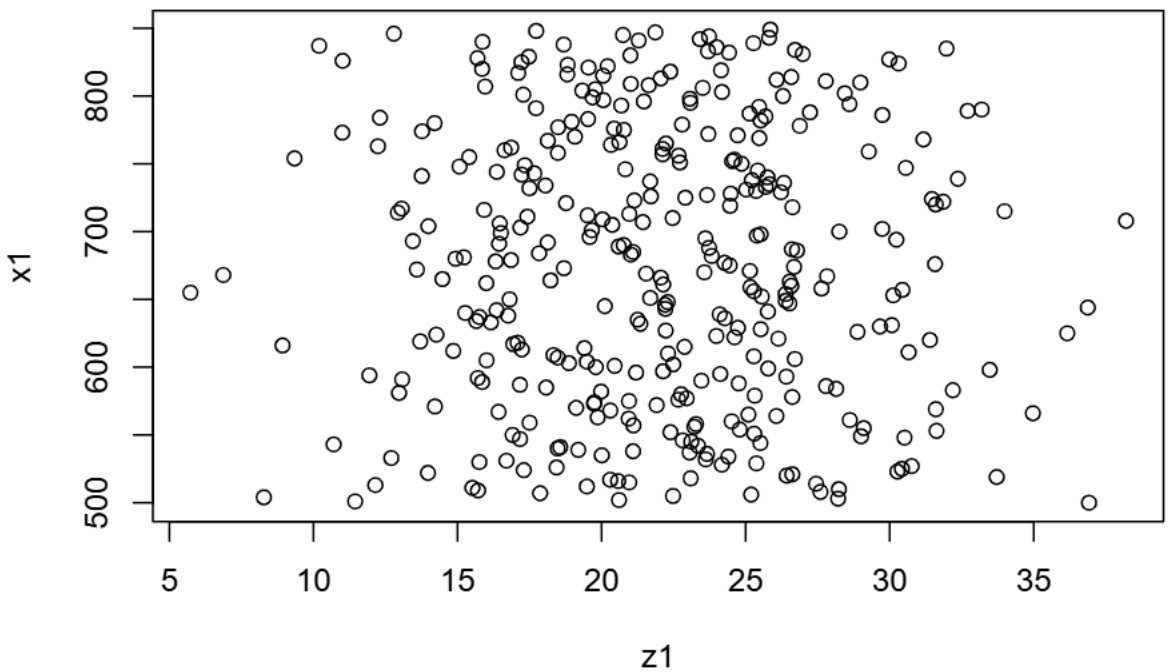
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
```

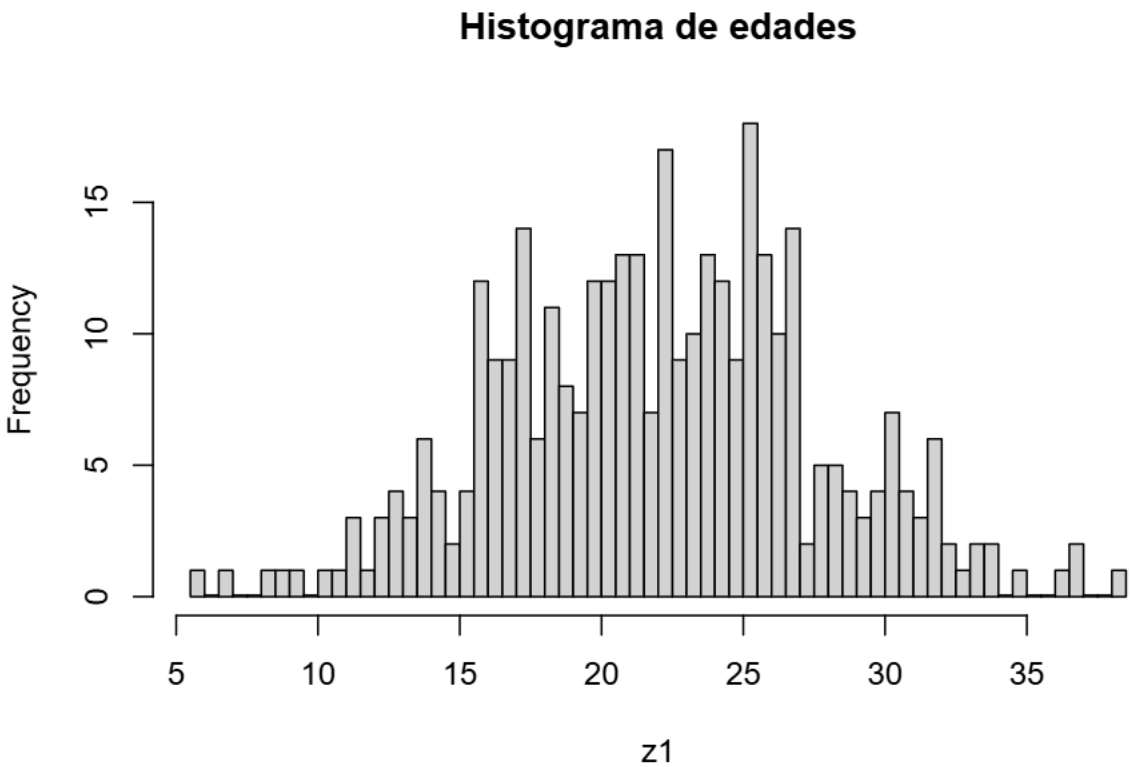
```
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando *hist()* genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



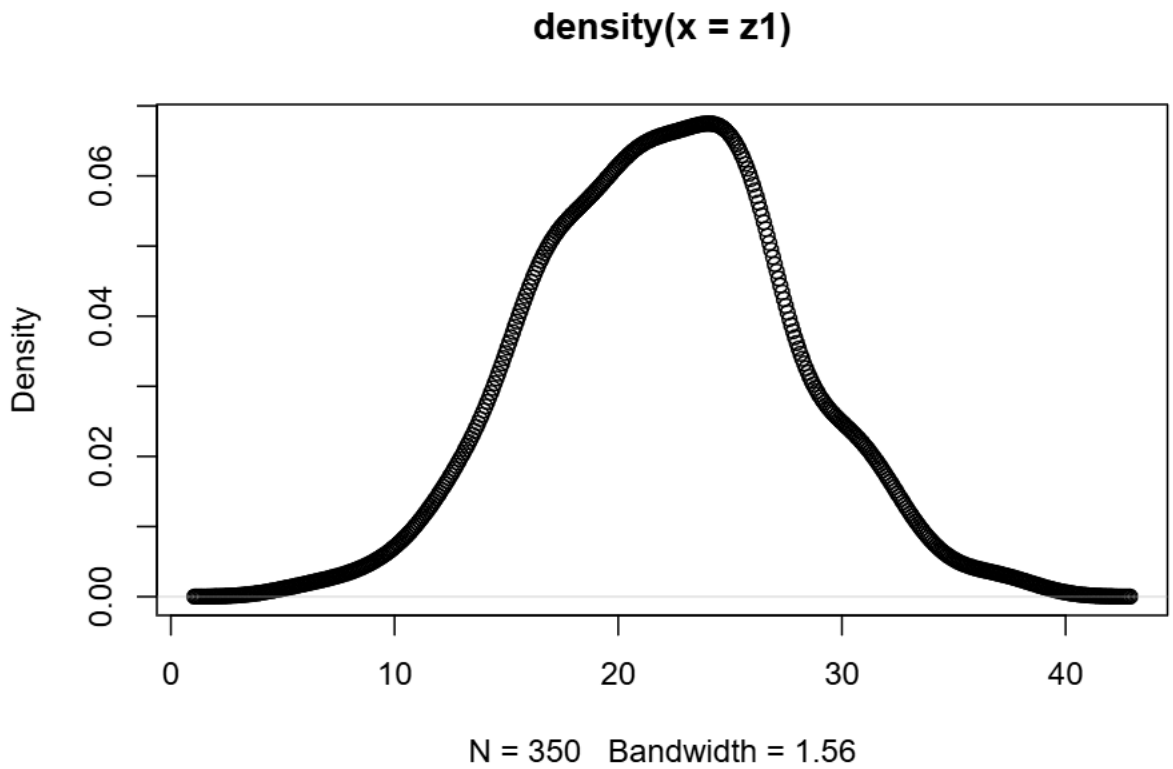
El comando *density()* calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.56
##
##        x                y
##  Min.   : 1.051   Min.   :8.903e-06
##  1st Qu.:11.509   1st Qu.:2.323e-03
##  Median :21.967   Median :1.358e-02
##  Mean   :21.967   Mean   :2.386e-02
##  3rd Qu.:32.425   3rd Qu.:4.753e-02
##  Max.   :42.883   Max.   :6.748e-02
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese número

```
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
```

```
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector *secuencia_dni* usando un for:

```r
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total
```

```
## [1] 322806
```

Ejercicio 3: Phyton

Consigna: Repetir el ejercicio anterior pero en Phyton

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (803)
dni = 803

# Consigna: Crear un vector (lista) del 1 al 803
# En Python, range(1, 804) genera números del 1 al 803
secuencia_dni = list(range(1, dni + 1))
```

```
# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")
```

```
## La suma total es: 322806
```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna:¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando **sys.time**

```
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 3.224045e+19
```

```
final <- Sys.time()
final-inicio
```

```
## Time difference of 2.245695 mins
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando **tictoc**

```
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}
# Fin del bloque a medir

#Se "detiene" el cronómetro.
```

```r
#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.004 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total
```

```
## [1] 322806
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for
```

```
## Time difference of 0.006142616 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.001807213 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
```

```
}

fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```
iteraciones
```

```
## [1] 32
```

Ejercicio 8: Definición matemática recurrente

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
```

```r
  }
  return(x)
}
```

Comparación:

```r
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  20.818   0.040  20.962
```

```r
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```r
n <- 1000000

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.018   0.000   0.018
```

Método 2 (fórmula matemática):

```r
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```r
suma1
```

```
## [1] 500000500000
```

```r
suma2
```

```
## [1] 500000500000
```

Conclusión final:

A lo largo de este trabajo, evaluamos la eficiencia de distintos algoritmos en R mediante técnicas de benchmarking (como Sys.time, tictoc y microbenchmark). Comprobamos de manera empírica que un código no es de calidad solo por llegar al resultado correcto, sino por cómo optimiza el uso del procesador y el tiempo de ejecución.Por ejempolo, el ejercicio de "La penitencia de Newton" demostró este principio claramente.

Mientras un bucle iterativo requiere un costo computacional inmenso para realizar operaciones masivas, aplicar un buen método matemático resuelve el mismo problema de forma casi instantánea.

La medición de tiempos que utilizamos lo confirma: la herramienta de optimización más poderosa en la programación no es el hardware, sino el razonamiento físico y matemático previo. Los modelos analíticos demuestran ser siempre abrumadoramente más eficientes que la iteración tradicional.